

matrix exponentiation

```

const int mod = 1e9 + 7;
using Row = vector<long long>;
using Matrix = vector<Row>;

Matrix mul(Matrix &a, Matrix &b) {
    int n = a.size(), m = a[0].size(), k = b[0].size();
    Matrix res(n, Row(k));
    for(int i = 0; i < n; ++i)
        for(int j = 0; j < k; ++j)
            for(int o = 0; o < m; ++o) {
                res[i][j] += 1ll * a[i][o] * b[o][j] % mod;
                if(res[i][j] >= mod) res[i][j] -= mod;
                if(res[i][j] < 0) res[i][j] += mod;
            }
    return res;
}

Matrix power(Matrix a, long long b) {
    int n = a.size();
    Matrix res(n, Row(n));
    for(int i = 0; i < n; ++i) res[i][i] = 1;

    while(b) {
        if(b&1) res = mul(res, a);
        a = mul(a, a), b >>= 1;
    }

    return res;
}

```

KMP Automaton

```

vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i-1];
        while (j > 0 && s[i] != s[j])
            j = pi[j-1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}

bool match(string s, string t) {
    string cur = t + '#' + s;

```

```

vector<int> pi = prefix_function(cur);

for (int &i : pi)
    if ( i == t.size())
        return true;
return false;
}

const int N=1e3;
int nxt[N][26];
void build_automaton(string& v) {
    int m = v.size();
    vector<int> pi(m);
    for (int i = 1; i < m; i++) {
        int j = pi[i-1];
        while (j > 0 && v[i] != v[j]) j = pi[j-1];
        if (v[i] == v[j]) j++;
        pi[i] = j;
    }
    for (int i = 0; i < m; i++) {
        for (int c = 0; c < 26; c++) {
            if (v[i] == (char)('A' + c)) nxt[i][c] = i + 1;
            else if (i > 0) nxt[i][c] = nxt[pi[i-1]][c];
            else nxt[i][c] = 0;
        }
    }
}

```

example of using KMP automaton

```

void solve()
{
    string s,t;
    cin>>s>>t;
    int n = s.size();
    int m = t.size();

    vector<int>pi(m+1,0);
    vector<vector<int>>nxt(m+1,vector<int>(26,0));

    for(int i=1;i<m;i++){
        int j = pi[i-1];
        while(j>0 && t[j]!=t[i])
            j = pi[j-1];
        if(t[i] == t[j]){
            j++;
        }
        pi[i] = j;
    }
}

```

```

for(int j=0;j<=m;j++){
    for(int c=0;c<26;c++){
        if(j<m && t[j] - 'a'== c){
            nxt[j][c] = j + 1;
        }else{
            if(j == 0)
                nxt[j][c] = 0;
            else
                nxt[j][c] = nxt[pi[j-1]][c];
        }
    }
}

vector<int>cur(m+1,-1);
cur[0] = 0;

int cnt = 0;
for(int i=0;i<n;i++){
    vector<int>nextDp(m+1,-1);
    for(int j=0;j<=m;j++){
        if(~cur[j]){
            if(s[i]=='?'){
                for(int c = 0;c<26;c++){
                    int to = nxt[j][c];
                    nextDp[to] = max(nextDp[to], cur[j] + (to == m));
                }
            }else{
                int c = s[i] - 'a';
                int to = nxt[j][c];
                nextDp[to] = max(nextDp[to], cur[j] + (to == m));
            }
        }
    }
    cur = nextDp;
}

for(int i=0;i<=m;i++){
    cnt = max(cur[i],cnt);
}
cout<<cnt<<'\n';
}

```

bellman ford

```

struct Edge {
    int u, v;
    long long w;
};

```

```

void solve() {
    int n , m;
    cin >> n >> m;
    vector<Edge> edges(m);
    for (int i = 0; i < m; ++i)
        cin >> edges[i].u >> edges[i].v >> edges[i].w;
    // source 0 linked with each nodes
    vector<ll> dist(n + 1, 0);

    for (int i = 1; i < n; ++i) {
        bool changed = false;
        for (const auto& e : edges) {
            if (dist[e.v] > dist[e.u] + e.w) {
                dist[e.v] = dist[e.u] + e.w;
                changed = true;
            }
        }
        if (!changed)
            break;
    }

    for (Edge e : edges)
        if (dist[e.v] > dist[e.u] + e.w)
            return void(cout << "-inf\n");

    ll min_dist = 1e18;
    for (int i = 1; i <= n; ++i)
        min_dist = min(min_dist, dist[i]);

    cout << min_dist << endl;
}

```

Trie pointers

```

struct NODE {

    NODE *children[26];
    int cntEndWith = 0;
    int cntPrefix = 0;
    bool is_end=false;
};

class Trie {
private:
    NODE* root;

public:
    Trie() { //constructor
        root = new NODE();
    }
    void insert(const string& word) {

```

```
    NODE* node = root;
    for (auto c:word)
    {
        if (node->children[c - 'a'] == NULL)//check if exist
            node->children[c - 'a'] = new NODE();

        node = node->children[c - 'a'];
        node->cntPrefix++;
    }
    node->cntEndWith++;
    node->is_end = true;
}

bool search(const string& word) {
    NODE* node = root;
    for (char c : word) {
        if (node->children[c - 'a'] == NULL)
            return false;
        node = node->children[c - 'a'];
    }
    return node->is_end;
}

bool startWith(const string& word) {
    NODE* node = root;
    for (char c : word) {
        if (node->children[c - 'a'] == NULL)
            return false;
        node = node->children[c - 'a'];
    }
    return true;
}

int CountWordsEqualTo(const string& word) {
    NODE* node = root;
    for (auto c : word) {
        if (node->children[c - 'a'] == NULL)
            return 0;
        node = node->children[c - 'a'];
    }
    return node->cntEndWith;
}

int CountWordsStartWith(const string& word) {
    NODE* node = root;
    for (auto c : word) {
        if (node->children[c - 'a'] == NULL)
            return 0;
        node = node->children[c - 'a'];
    }
    return node->cntPrefix;
}

void erase(string s) {
    NODE* node = root;
    for (auto c : s) {
        if (node->children[c - 'a'] == NULL)
            return;
        node = node->children[c - 'a'];
    }
}
```

```

        node->cntPrefix--;
    }
    node->cntEndWith--;
}
};

```

Trie vector

```

class Trie {
private:
    struct Node {
        vector<int> next;
        bool is_end;
        // Count how many words pass through this node
        int pass_count;

        Node() : next(26, 0), is_end(false), pass_count(0) {}
    };

    vector<Node> trie;

public:
    Trie() {
        trie.emplace_back(); // Root
    }

    void insert(const std::string& word) {
        int curr = 0;
        for (char c : word) {
            int idx = c - 'a';
            if (trie[curr].next[idx] == 0) {
                trie[curr].next[idx] = trie.size();
                trie.emplace_back();
            }
            curr = trie[curr].next[idx];
            trie[curr].pass_count++; // Track usage
        }
        trie[curr].is_end = true;
    }

    bool erase(const std::string& word) {
        if (!search(word)) return false; // Word doesn't exist

        int curr = 0;
        for (char c : word) {
            int idx = c - 'a';
            int next_node = trie[curr].next[idx];

            trie[next_node].pass_count--;

            // Optimization: If no more words use this path,

```

```

        // we can "sever" the link to prune the branch.
        if (trie[next_node].pass_count == 0) {
            trie[curr].next[idx] = 0;
            return true;
        }
        curr = next_node;
    }

    trie[curr].is_end = false;
    return true;
}

bool search(const std::string& word) {
    int curr = 0;
    for (char c : word) {
        int idx = c - 'a';
        if (trie[curr].next[idx] == 0) return false;
        curr = trie[curr].next[idx];
    }
    return trie[curr].is_end;
}
};

```

Hashing

```

const int mod = 1e9 + 7;
const int N = 1e6;
pair<int, int> pw[N], inv[N];
int add(int a, int b) {
    return (1ll * a + b + mod) % mod;
}
int mul(int a, int b) {
    return (1ll * a * b) % mod;
}
int fastpow(int a, int b) {
    if (!b) return 1;
    int hp = fastpow(a, b >> 1);
    hp = mul(hp, hp);
    return (b & 1 ? mul(hp, a) : hp);
}
int modInverse(int a) {
    return fastpow(a, mod - 2);
}
void preHash() {
    vector<int> primes;
    for (int i = 1; i <= 293; i++) {
        bool p = true;
        for (int j = 2; j * j <= i; j++) {
            if (i % j == 0) p = false;
        }
    }
}

```

```

        if (p) primes.push_back(i);
    }
    srand(time(0));
    int b1 = primes[rand() % 62], b2 = primes[rand() % 62];
    pw[0] = inv[0] = {1, 1};
    int invB1 = modInverse(b1), invB2 = modInverse(b2);
    for (int i = 1; i < N; i++) {
        pw[i] = {mul(b1, pw[i - 1].first), mul(b2, pw[i - 1].second)};
        inv[i] = {mul(invB1, inv[i - 1].first), mul(invB2, inv[i - 1].second)};
    }
}

struct Hash {
    vector<pair<int, int>> pre;
    pair<int, int> h(int num, int p) {
        return {mul(num, pw[p].first), mul(num, pw[p].second)};
    }
    int fa7l(char a) {
        return (a - 'a' + 1);
    }
    Hash(string &s) {
        pre.resize(s.size());
        for (int i = 0; i < s.size(); i++) {
            pre[i] = h(fa7l(s[i]), i);
            if (i) pre[i] = {add(pre[i].first, pre[i - 1].first),
add(pre[i].second, pre[i - 1].second)};
        }
    }
    pair<int, int> getPrefix(int l, int r) {
        if (!l) return pre[r];
        return {mul(add(pre[r].first, -pre[l - 1].first), inv[l].first),
mul(add(pre[r].second, -pre[l - 1].second), inv[l].second)};
    }
};

```

Sparse Table

```

ll myFunction(ll a, ll b) {
    return a;
}

struct SparseTable {
    vector<vector<ll>> table;
    vector<int> lgs;

    SparseTable(vector<ll>& arr) {
        int n = arr.size();
        table = vector<vector<ll>>(n + 1, vector<ll>(20));
        lgs = vector<int>(n + 1);

        for (int i = 0; i < n; i++)
            table[i][0] = arr[i];
    }
};

```

```

        for (int i = 2; i <= n; i++)
            lgs[i] = lgs[i / 2] + 1;
        for (int lg = 1; lg <= lgs[n]; lg++)
            for (int i = 0; i + ((1ll << lg) - 1) < n; i++)
                table[i][lg] =
                    myFunction(table[i][lg - 1], table[i + ((1ll << (lg - 1)))[lg -
1]]);
    }

    ll get(int l, int r) {
        return myFunction(table[l][lgs[r - l + 1]],
                           table[r - ((1 << lgs[r - l + 1]) + 1)[lgs[r - l + 1]]]);
    }
};

```

Digit DP segment sum problem

```

const int mod = 998244353;
ll add(ll a, ll b) {
    return (a + b + mod) % mod;
}
ll mul(ll a, ll b) {
    return (a * b) % mod;
}
struct Node{
    ll cnt, sum;
};
bool memo[2][2][(1<<10)][20];
Node dp [2][2][(1<<10)][20]; //ru,rd,lz,msk (taken digits) , i = n
string l,r;
int n,k;

ll pow10[20];
Node rec(bool ru,bool rd,int msk,int i){

    if(i == n){
        //
        return {1,0};
    }

    Node &ret = dp[ru][rd][msk][i];
    if(memo[ru][rd][msk][i])
        return ret;

    short up = ru ? r[i] - '0' : 9;
    short down = rd ? l[i] - '0' : 0;
    ret = { 0 , 0};
    for(ll d=down ; d<=up ; d++){
        ll nexMsk = msk;
        if( !(msk == 0 && d == 0)) // not leading zeros
            nexMsk = msk | (1<<d);
    }
}

```

```
        if( __builtin_popcount(nexMsk) <= k ){
            Node temp = rec(ru && d==up , rd && d==down , nexMsk ,i+1);

            ret.cnt = add(ret.cnt , temp.cnt);
            ll dig_val = mul(mul(d , pow10[n-i-1]) , temp.cnt );
            ret.sum = add ( add(ret.sum,dig_val) , temp.sum);
        }
    }
    memo[ru][rd][msk][i] = true;
    return ret;
}

void solve() {
    cin>>l>>r >> k;
    n = r.size();
    while(l.size() != n)
        l = "0" + l;

    memset(dp,-1,sizeof dp);

    pow10[0] = 1;
    for(int i=1;i<20;i++)
        pow10[i] = mul(pow10[i-1],10);

    cout<<rec(1,1,0,0).sum <<'\n';
}
```